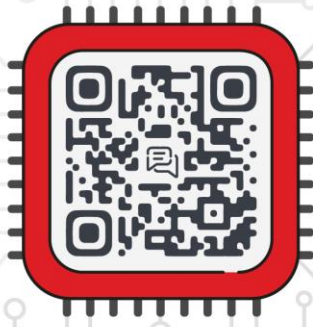




عملي

13

1

1950 sp

كلية الهندسة المعلوماتية

السنة الرابعة

Concurrent Access &
Thread Safety

عملي مشترك



محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 07/04/2026

برمجة تفرعية

تُعدّ مادة **البرمجة التفرعية** امتداداً طبيعياً لما درسناه في مقررات البرمجة السابقة (برمجة 1، 2، 3...). ففي تلك المقررات، لم يكن الهدف تعلّم لغة برمجة بحدّ ذاتها، بل التركيز على **المفاهيم الأساسية** مثل المتغيرات، الشروط، الحلقات، الدوال، وهياكل البيانات، ثم تطبيق هذه المفاهيم باستخدام لغة معينة.

وبنفس الأسلوب، سنسير في هذه المادة؛ حيث لن نركّز على لغة محددة، بل على **مفاهيم وأساليب جديدة** يمكن تطبيقها على أي لغة تدعم البرمجة التفرعية.

لماذا تُسمّى "برمجة تفرعية"؟

يشير مصطلح Parallel Programming إلى أسلوب برمجي يتم فيه تنفيذ عدة أجزاء من البرنامج في نفس الوقت (بالتوازي) بدلاً من تنفيذها بشكل متسلسل (واحدًا بعد الآخر).

بمعنى آخر:

▪ في البرمجة التقليدية:

يتم تنفيذ الأوامر **خطوة بخطوة**.

▪ في البرمجة التفرعية:

يمكن تنفيذ عدة **مهام في نفس اللحظة** باستخدام ما يُعرف بـ **الخيوط (Threads)** أو العمليات (Processes).

العلاقة مع المقررات السابقة:

في المقررات السابقة، كان تركيزنا الأساسي على:

1. المتطلبات الوظيفية (Functional Requirements)

أي: ماذا يفعل البرنامج؟

مثل: حساب مجموع، إدارة بيانات، تنفيذ عملية معينة...

أما في هذه المادة، فسنستجّه نحو:

2. المتطلبات غير الوظيفية (Non-Functional Requirements)

أي: كيف يعمل البرنامج؟ وبأي كفاءة؟

ومن أهم هذه الجوانب:

- الأداء (Performance) : هل البرنامج سريع؟
- الحمولة (Load Handling) : هل يستطيع التعامل مع عدد كبير من الطلبات؟
- التزامن (Concurrency) : هل يمكن تنفيذ عدة عمليات في نفس الوقت؟
- المصادر المشتركة (Shared Resources) : كيف تتشارك العمليات نفس البيانات دون حدوث مشاكل

الفكرة الأساسية للمادة:

لم يعد السؤال فقط:

"هل الكود يعمل بشكل صحيح؟"

بل أصبح:

"هل الكود قادر على العمل بكفاءة عند وجود عدة عمليات أو خيوط تعمل في نفس الوقت؟"

أي أننا سنعمل على:

- تطوير الكود ليصبح قادراً على التعامل مع التوازي
- فهم كيف يمكن أن تؤثر عدة خيوط على نفس البيانات
- تحليل سلوك البرنامج تحت الضغط

التزامن (Concurrency)

تمهيد للفكرة (مثال المطبخ)

لنفترض أننا في مطبخ، ولدينا 3 طاولات، وكل طاولة لديها طلب مختلف.
هنا يوجد طريقتان للعمل:

الطريقة الأولى (تنفيذ تسلسلي):

نقوم بتنفيذ الطلب الأول كاملاً، ثم ننتقل إلى الطلب الثاني، ثم الثالث.
أي نعمل على مهمة واحدة في كل مرة.

الطريقة الثانية (تنفيذ متزامن):

نبدأ بالطلب الأول، مثلاً نضع الماء ليغلي لتحضير الشوربة،
وخلال انتظار الغليان، ننتقل إلى الطلب الثاني ونسلق المعكرونة،
وأثناء ذلك، نبدأ بتقطيع الخضروات للطلب الثالث.
في هذه الحالة، نحن لا ننقذ المهام في نفس اللحظة تماماً،
ولكننا نقوم بالتبديل بينها وإدارتها بحيث تكون جميعها قيد التنفيذ.

■ ماذا نستنتج من المثال؟

بهذا الأسلوب:



- **قلّ** الوقت الإجمالي لإنهاء الطلبات.
- أصبحت **الاستجابة أسرع**.
- **زادت** الإنتاجية.
- تم **استغلال الموارد** المتاحة بشكل **أفضل**.



- ربط المثال بالحاسوب
- يمكن إسقاط هذا المثال على نظام الحاسوب كالتالي:
- المطبخ : وحدة المعالجة (CPU).
- الشيف : وحدة التحكم (Control Unit).
- الطلب : خيط التنفيذ (Thread).
- أي أن الحاسوب، مثل الشيف، يقوم بـ:

إدارة **عدة مهام** (Threads) **والتبديل بينها** لتحقيق أفضل أداء ممكن.

- مثال إضافي (من الواقع: صفحات الويب)

للتخيّل ما يحدث عند فتح صفحة ويب:

كل **مستخدم (Client)** يقوم بإرسال **طلب (Request)** إلى **الخادم (Server)**.

الآن، افترض أن هذا الخادم **غير قادر إلا على معالجة طلب واحد فقط في كل مرة (تنفيذ تسلسلي)**:

- سيقوم بخدمة المستخدم الأول.
- بينما ينتظر باقي المستخدمين في طابور طويل.
- ومع وجود آلاف أو حتى ملايين الطلبات، سيصبح:

- زمن الانتظار طويل جداً.
- الاستجابة بطيئة.
- أداء النظام سيئ للغاية.

تخيّل تطبيقاً مثل موقع حجز أو متجر إلكتروني يعمل بهذه الطريقة! سيكون غير قابل للاستخدام عملياً.

ماذا يحدث عند استخدام التزامن؟

إذا كان الخادم يدعم **الزامن (Concurrency)** :

- لن ينتظر حتى ينتهي من طلب كامل ليبدأ بآخر.
- بل سيقوم بإدارة **عدة طلبات في نفس الوقت**.
- ينتقل بين الطلبات:



■ يعالج جزءاً من الطلب الأول

■ ثم ينتقل للثاني

■ ثم الثالث... وهكذا

■ تماماً كما في مثال المطبخ:

■ لا ينتظر انتهاء مهمة بالكامل

■ بل يقوم بالتبديل بين المهام لإبقائها جميعاً قيد التنفيذ

➤ النتيجة:

■ باستخدام التزامن:

■ يقل زمن الانتظار.

■ تتحسن سرعة الاستجابة.

■ يستطيع النظام خدمة عدد أكبر من المستخدمين.

■ يصبح النظام أكثر كفاءة وقابلية للتوسع.

➤ من خلال مثال المطبخ والخدم، نلاحظ أن الفكرة الأساسية هي:

■ إدارة عدة مهام أو طلبات بحيث تكون جميعها قيد التنفيذ (in progress)، بدلاً من معالجة كل مهمة بشكل منفصل بالكامل.

■ وهذا يقودنا مباشرة إلى تعريف التزامن (Concurrency).

■ التزامن (Concurrency): هو قدرة النظام على إدارة عدة مهام في نفس الوقت، بحيث تكون جميعها قيد التنفيذ (In Progress)، حتى لو لم تُنفَّذ في نفس اللحظة تماماً.

■ ملاحظة هامة:

■ هذا يختلف عن التوازي (Parallelism)، الذي يعني تنفيذ المهام في نفس اللحظة فعلياً.

➤ لماذا نحتاج إلى التزامن؟

1. الاستجابة (Responsiveness)

■ يساعد في منع تجمّد التطبيقات أثناء تنفيذ العمليات الطويلة، مثل تحميل ملف أو إجراء عملية حسابية كبيرة.

2. الإنتاجية (Throughput)

■ يسمح للنظام بإنجاز كمية أكبر من العمل خلال وقت أقل.

3. استغلال الموارد (Resource Utilization)

■ يساعد في الاستفادة بشكل أفضل من قدرات المعالج، خاصة في الأنظمة الحديثة متعددة الأنوية (Multi-core).

■ التأكيد على الفرق بين التزامن (Concurrency) والتوازي/التفرعية (Parallelism) :

Concurrency vs Parallelism



من المهم جداً التمييز بين هذين المفهومين، لأن الخلط بينهما شائع:

التزامن (Concurrency) :

هو إدارة عدة مهام في نفس الوقت

(Managing multiple tasks at once)

أي أن النظام يتعامل مع عدة مهام ويقوم بالتبديل بينها بحيث تكون جميعها قيد التنفيذ (In Progress)، حتى لو لم تنفذ في نفس اللحظة الفعلية.

التوازي/التفرعية (Parallelism) :

هو تنفيذ عدة مهام في نفس الوقت فعلياً

(Executing multiple tasks at once)

أي أن المهام تنفذ بشكل متزامن حقيقي، غالباً باستخدام عدة أنوية (Cores) في المعالج. بالربط مع مثال المطبخ:

في التزامن	في التوازي/التفرع
لدينا شيف واحد يدير عدة طلبات ويبدل بينها.	لدينا عدة طهاة، وكل واحد يعمل على طلب مختلف في نفس اللحظة.

لا تضيق وقتك على وسائل التواصل الاجتماعي أو مشاهدة الأفلام؛ بل خصص وقتاً لهم.



مفهوم الموارد المشتركة (Shared Resources)

ما هو المورد المشترك؟

المورد المشترك هو أي بيانات أو مكوّن يمكن الوصول إليه من قبل أكثر من خيط (Thread) أو عملية (Process) في نفس الوقت

أمثلة شائعة:

- سجلات قاعدة البيانات
- الذاكرة المشتركة

- متغيرات عامة (Global Variables)
- ملفات على القرص
- مثال توضيحي :



لنفترض لدينا متغير مشترك:

```
int counter = 0 ;
```

ولدينا خيطان:

• Thread A

• Thread B

وكلاهما يريد تنفيذ العملية:

```
counter = counter + 1 ;
```

- ماذا يحدث فعلياً داخل كل Thread؟
- عملية الزيادة ليست خطوة واحدة، بل ثلاث خطوات:

1. قراءة القيمة من الذاكرة إلى متغير محلي (Local Copy)

2. تعديل القيمة (إضافة 1)

3. إعادة كتابة القيمة إلى المورد المشترك

- الحالة المثالية (بدون تنافس):

لو تم التنفيذ بشكل مرتب:

Thread A:

■ يقرأ $counter = 0$

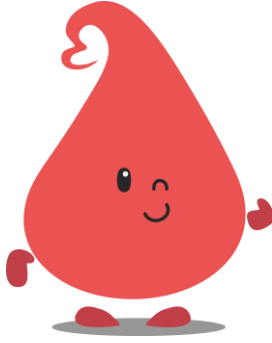
■ يزيد بها بمقدار 1

■ يكتبها $counter = 1$

ثم Thread B:

■ يقرأ $counter = 1$





- يزيد بها بمقدار 1
- يكتبها counter = 2
- النتيجة النهائية:
- counter = 2
- هذه هي النتيجة الصحيحة والمتوقعة.

■ المشكلة (عند التزامن بدون تنظيم):

ماذا لو حدث التداخل التالي:

- Thread A يقرأ counter = 0
- Thread B يقرأ counter = 0 (قبل أن يكتب A التحديث)
- Thread A يزيد 1 ويكتبها
- Thread B يزيد 1 على نسخته المحلية المساوية لـ 0 ويكتبها

■ النتيجة النهائية:

counter = 1

رغم أن هناك عمليتي زيادة، إلا أن النتيجة أصبحت خاطئة!

■ التحدي (The Challenge)

عندما تقوم عدة Threads بتعديل نفس المورد المشترك في نفس الوقت:

- قد يحدث تداخل في التنفيذ
- ويتم فقدان بعض التعديلات
- وتصبح النتيجة غير متوقعة (Unpredictable)

هذه المشكلة تعرف بمشكلة:

تعارض الوصول/حالة التنافس (Race Condition).

حالة التنافس Race Condition

تحدث حالة التنافس (Race Condition) عندما:

تعتمد نتيجة البرنامج على التوقيت غير المتوقع (Non-deterministic) أو على ترتيب تنفيذ (Interleaving) الخيوط (Threads)

وسُمّيت "Race" لأن:

الخيوط تكون كأنها تتسابق للوصول إلى نفس المورد المشترك، والخيوط الذي "يفوز" في النهاية هو من يحدد النتيجة. لماذا تُعتبر خطيرة؟

تكمّن خطورة الـ Race Condition في أنها:

- لا تحدث دائماً
- تعتمد على توقيت معيّن يصعب التحكم به
- تؤدي إلى أخطاء متقطعة (Intermittent Bugs)
- من الصعب جداً إعادة إنتاجها (Hard to Reproduce)

وهذا يجعل اكتشافها وتصحيحها من أصعب المشاكل في البرمجة التفرعية.

فائدة أربيسيزية : Debugging Nightmare/كابوس التصحيح

تُوصف حالات التنافس (Race Conditions) بأنها كابوس في التصحيح (Debugging Nightmare) لأنها لا تحدث بشكل ثابت، بل تعتمد على توقيت تنفيذ الخيوط (Threads) الذي قد يتغيّر في كل تشغيل للبرنامج.

فقد يعمل البرنامج بشكل صحيح عشرات المرات، ثم يظهر الخطأ فجأة في ظروف معيّنة يصعب تكرارها.

المشكلة الأساسية هي:

- صعوبة إعادة إنتاج الخطأ (Reproducing the Bug)
 - لأن ترتيب التنفيذ (Interleaving) يتغير باستمرار
 - وقد يختفي الخطأ عند محاولة تتبّعه (مثلاً عند إضافة Print أو Debugger)
- لذلك، يصبح المطوّر أمام حالة:

الخطأ موجود، لكن لا يمكن الإمساك به بسهولة أو فهم متى ولماذا يحدث وهذا ما يجعل هذه الحالات تُشبه “الكابوس” أثناء عملية التصحيح.

■ دورة القراءة-التعديل-الكتابة (Read-Modify-Write)

معظم حالات التنافس تحدث ضمن تسلسل شائع جداً يتكون من 3 خطوات:

1. Read قراءة:

يقوم الـ Thread بقراءة القيمة من الذاكرة

2. Modify تعديل:

يقوم بتعديل القيمة محلياً (مثل: إضافة 1)

3. Write كتابة:

يعيد كتابة القيمة الجديدة إلى الذاكرة

■ أين تحدث المشكلة؟

المشكلة تحدث إذا:

قام Thread آخر بعملية Write بين عمليتي Read و Write لل Thread الأول في هذه الحالة:

- سيتم استبدال التعديل السابق.
- وستفقد بعض التحديثات.

مثال داعم: مشكلة الحساب البنكي (Bank Account Problem)

الحالة الابتدائية

لدينا حساب بنكي مشترك:

الرصيد = \$100

وهناك مستخدمان يحاولان السحب في نفس الوقت:

■ User A: يريد سحب \$60

■ User B: يريد سحب \$60

النتيجة المتوقعة:

■ تنجح عملية واحدة فقط

■ وتفشل الثانية بسبب عدم كفاية الرصيد (لأن المجموع المطلوب = \$120 < \$100)



✎ ماذا يحدث فعلياً بدون تنظيم (Race Condition) ؟

لننظر إلى تسلسل التنفيذ بين الخيطين:

الزمن	Thread A (سحب \$60)	Thread B (سحب \$60)	الرصيد
T1	يقرأ الرصيد(100)		100
T2		يقرأ الرصيد(100)	100
T3	يحسب: $40 = 60 - 100$		100
T4		يحسب: $40 = 60 - 100$	100
T5	يكتب الرصيد(40)		40
T6		يكتب الرصيد(40)	40 X corrupt

لا تنتظر للوراء، لأنك لن تذهب إلى ذلك الطريق



تحليل المشكلة

كل Thread قام بعملية $\text{Read} \rightarrow \text{Modify} \rightarrow \text{Write}$ بشكل مستقل
لكن :

Thread B قرأ الرصيد قبل أن ينهي Thread A التحديث.

أي أنه عمل على بيانات قديمة (Stale Data) .

النتيجة

الرصيد النهائي = 40\$ وهذا خطأ! لأن:

تم تنفيذ عمليتي سحب ($60 + 60 = 120$)

لكن الرصيد لم يعكس ذلك

تُعرف هذه الحالة بـ: (**Lost Update فقدان التحديث**)

حيث تم تجاهل أحد التحديثات وكأن إحدى العمليتين لم تحدث.

تعريف (Thread Safety) سلامة الخيوط

ما المقصود بـ Thread Safety ؟

يُقال عن جزء من الكود أنه Thread-Safe إذا:

يعمل بشكل صحيح عند تنفيذه من قبل عدة Threads في نفس الوقت، دون أن يؤدي ذلك إلى نتائج خاطئة أو تأثيرات جانبية غير مرغوبة

بمعنى آخر:

يحافظ على صحة البيانات (Integrity).

ويعطي نتائج صحيحة ومتوقعة دائماً.

حتى في وجود تنفيذ متزامن.

مقارنة بين الكود الآمن وغير الآمن:

الخاصية	Thread-Safe Code	Non-Thread-Safe Code
الوصول للبيانات المشتركة	يتم بشكل منظم (Synchronization) أو البيانات غير قابلة للتغيير (Immutable)	يتم التعديل من عدة Threads بدون أي تنظيم
نتيجة التنفيذ	دائماً متوقعة وثابتة	تعتمد على توقيت التنفيذ (Race Condition)
الحالة الداخلية (State)	تبقى سليمة ومتناسقة (Maintains integrity) (at all times)	قد تصبح فاسدة أو غير متناسقة
استخدام الموارد	يتم التحكم به عن طريق (Locks / Semaphores / Atomic Operations)	وصول عشوائي يؤدي إلى تعارض

أساسيات (Synchronization) التزامن المنظم

Synchronization المزامنة هو:

تنسيق عمل عدة Threads بحيث لا تتداخل مع بعضها عند الوصول إلى الموارد المشتركة
أي أننا لا نمنع التزامن، بل ننظمه بحيث نحافظ على صحة البيانات ونتجنب الأخطاء.

➤ **المبدأ الأساسي: مبدأ الإقصاء المتبادل (Mutual Exclusion /Mutex)**
الفكرة الأساسية هي:

يسمح لخيط (Thread) واحد فقط بالدخول إلى جزء معين من الكود (نسميه critical area) أو استخدام مورد معين في لحظة واحدة.

The Impact of Synchronization تأثير المزامنة:

الحالة	بدون مزامنة (Unsynchronized)	مع مزامنة (Synchronized)
وصول الخيوط (Threads)	الخيوط تتسابق للوصول إلى المورد	الخيوط تنتظر دورها
سلامة البيانات	احتمال كبير لفساد البيانات	يتم الحفاظ على سلامة البيانات
نتائج التنفيذ	نتائج غير متوقعة	نتائج ثابتة وموثوقة
الأداء	أسرع (لا يوجد انتظار)	أبطأ قليلاً (بسبب وقت الانتظار)

➤ **كيف تعمل الأقفال : دورة الحياة (Locks Lifecycle)**

1. الخطوة 1: Acquire الحصول على القفل

يحاول الـ Thread الحصول على القفل (Lock) :

- إذا كان القفل **متاحاً** = يحصل عليه مباشرة.
- إذا كان القفل **مستخدماً من قبل Thread آخر** = يجب عليه **الانتظار** (Block) حتى يتم تحريره.

2. الخطوة 2: Process التنفيذ داخل القسم الحرج

بعد الحصول على القفل:

■ يدخل الـ Thread إلى ما يُسمى **القسم الحرج** (Critical Section)

■ يمكنه الآن:

- قراءة البيانات المشتركة
- تعديلها دون أي تداخل من Threads أخرى

3. الخطوة 3: Release تحرير القفل

بعد الانتهاء:

■ يقوم الـ Thread بـ **تحرير القفل**.

■ مما يسمح لباقي Threads المنتظرة بالحصول عليه.

قاعدة مهمة جدًا:

كل عملية Acquire يجب أن يقابلها Release وإلا سيبقى المورد مغلقاً للأبد.

مقدمة إلى Deadlocks

يحدث Deadlock عندما:

تتوقف مجموعة من Threads بشكل دائم، بحيث ينتظر كل منها مورداً يحتفظ به Thread آخر، أي لا يمكن لأي منها التقدم، فيتوقف البرنامج.

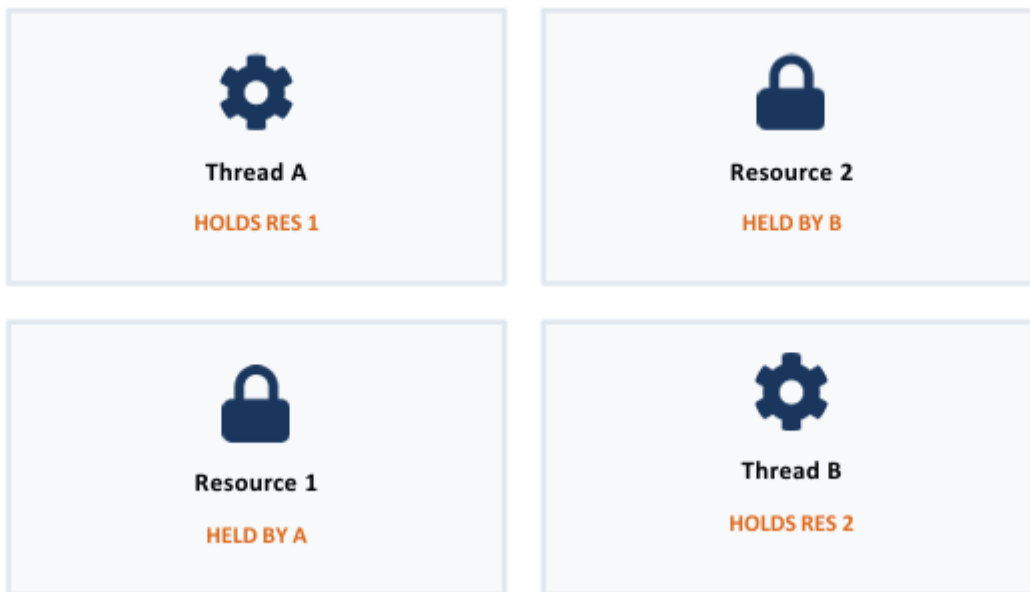
لماذا هو خطير؟

- البرنامج يتوقف عن التقدم (No Progress).
- لا يوجد حل داخلي.
- غالباً يحتاج إلى إعادة تشغيل النظام أو التطبيق.
- لماذا يُسمى "Deadly Embrace"؟

يُشَبَّه بـ:

عناق قاتل حيث كل Thread "يمسك" مورداً وينتظر الآخر، ولا أحد يستطيع التحرك أو التراجع.

↪ سيناريو الانتظار الدائري (Circular Wait):



لنفترض:

- Thread A يملك Resource 1.
- Thread B يملك Resource 2.

ثم:

- Thread A ينتظر Resource 2.
- Thread B ينتظر Resource 1.

النتيجة:

- لا أحد يستطيع التقدّم.
- ولا أحد سيحرّر المورد الذي لديه.

و منه يحدث توقّف كامل للنظام.

أفضل الممارسات لتحقيق Thread Safety

1. تقليل مشاركة البيانات (Minimize Shared State)

أفضل طريقة لتجنب مشاكل التزامن هي عدم مشاركة البيانات أساساً

- استخدم متغيرات محلية local بدل Global / static عند الإمكان

2. تصغير القسم الحرج (Keep Critical Sections Small)



- لا تضع Lock على كود كبير.
- اجعل الجزء المحمي صغيراً قدر الإمكان.

هذا يساعد على:

- تحسين الأداء.
- تقليل احتمال حدوث Deadlock.

3. استخدام كائنات غير قابلة للتغيير (Immutable Objects)

الكائن الذي لا يمكن تعديله بعد إنشائه هو آمن تلقائياً

- لا يحتاج إلى Locks.
- لا يسبب تعارض.

استخدم بيانات للقراءة فقط (Read-Only) قدر الإمكان

4. استخدام أدوات عالية المستوى (High-Level Utilities)

- تجنّب الأدوات منخفضة المستوى مثل: wait / notify
- استخدم مكتبات جاهزة مثل: Java: java.util.concurrent

لأنها:

- مجرّبة - موثوقة - تقلّل الأخطاء.

انتهت المحاضرة